Software Requirements Specification

Zoba — A Group Trip Planner

Group 2 | Kiefer Lin, Ashley Chen, John Yoon, Andrew Ke, Lauren Lee, Dylan Zhao

Version 1.0 • October 12, 2026

Source: Project Proposal dated September 10, 2026

Status: Draft for team review. “Proposed” items are clarifications, not commitments in the source proposal.

1. Introduction

1.1 Purpose

Define verifiable software requirements for Zoba, a mobile-responsive web application that aggregates group travel preferences and generates feasible multi-day itineraries with trip cost and preparation tools. Intended readers: project team, instructors, testers, and reviewers.

1.2 Scope

For groups of 3–8 travelers, Zoba supports trip creation, invitation, preference allocation and pairwise comparisons, fairness-aware group ranking, automated scheduling, map/timeline display, preparation tasks, expense estimation, and shared-expense settlement calculations. Booking/checkout, payment transfers, and native mobile applications are excluded. Venue booking may link to external sites.

1.3 Terms

Term	Meaning
Organizer	Member responsible for trip setup and constraint configuration.
Member	Invited trip participant who provides preferences and views plans.
Hard no	Destination vetoed by a member.

Bradley–Terry	Model that estimates relative preference from pairwise comparisons.
OSRM / OTP	Open Source Routing Machine / OpenTripPlanner.
CP-SAT	Constraint programming solver in Google OR-Tools.

2. Overall Description

2.1 Users and operating environment

Organizers configure city, dates, daily start/end windows and home base, invite participants, manage members and regenerate schedules. Members allocate preference points, complete comparisons, review the itinerary, and use trip-preparation tools. The proposal also permits a member to lock a stop; authority and conflict rules require clarification. Modern JavaScript-enabled desktop and mobile browsers are required; no camera or location permission is required.

2.2 Primary workflow

Create account → create trip → join via invite link → select/rank destinations or allocate points → complete adaptive this-or-that comparisons → calculate individual scores and fairness-aware group ranking → generate and compare feasible schedules → display map/timeline → estimate costs and create tasks → record shared expenses and calculate settlement.

2.3 Dependencies and constraints

Place data: Foursquare Open Source Places, OpenStreetMap/Overpass; descriptions: Wikivoyage/Wikidata. Routing: self-hosted OSRM, optionally OpenTripPlanner using GTFS for selected cities. Storage: PostgreSQL/PostGIS. Backend: Python 3/FastAPI with NumPy/SciPy. Optimization: Google OR-Tools CP-SAT; frontend: React/Vite/MapLibre GL; deployment: Docker Compose. Travel-time matrices are cached before solving; opening-hours data requires defensive parsing and curation. Solver uses a configured time limit and returns its best feasible result.

3. Functional Requirements

Identifiers follow the source proposal exactly; F05 was not defined. Each “shall” statement operationalizes the proposal. Acceptance checks describe observable outcomes, not invented numeric targets.

Account and trip setup

F01 — The system shall allow users to register and log in using email.

Verification: A registered user can authenticate and access their account.

F02 — The system shall allow an organizer to create a trip with city, date range, daily start/end times, and home base.

Verification: A saved trip displays the entered configuration and can be reopened.

Preference collection and consensus

F03 — The system shall present two candidate destinations at a time and record the member's selection.

Verification: A submitted choice is stored with the member and destination pair.

F04 — The system shall estimate each member's destination preference scores from comparisons using a Bradley–Terry model and select informative subsequent pairs.

Verification: Recorded comparisons produce per-member scores and subsequent pairs can be generated.

F06 — The system shall combine normalized member scores into a group ranking using a fairness-aware objective that considers the worst individual outcome.

Verification: A group ranking can be produced without rewarding a member merely for completing more comparisons.

F07 — The system shall include each member's highest-ranked eligible destination in the itinerary.

Verification: Each member's top eligible destination appears, or an infeasibility/conflict is reported (proposed exception handling).

F07a — Before pairwise voting, the system shall allow members to rank desired destinations or allocate a fixed point budget.

Verification: Submitted ranks or points persist and are associated with the member.

F08 — The system shall exclude any destination marked "hard no" by a member from scheduling.

Verification: A vetoed destination is absent from generated itineraries.

F08a — The system shall compute capped preference weights from F07a inputs and display raw vote counts alongside weighted scores.

Verification: The interface shows both metrics and applies the configured cap.

Itinerary generation

F09 — The system shall select and order destinations across trip days to maximize the group preference objective subject to constraints.

Verification: Generation produces an ordered itinerary with destination assignments.

F10 — The system shall respect opening hours, inter-stop travel time, and daily time windows.

Verification: No scheduled stop violates known opening or daily windows, and travel intervals are accounted for.

F11 — The system shall implement both a fast greedy heuristic and an OR-Tools-based solver and record comparable quality and runtime measures.

Verification: Both methods run on the same trip inputs and produce recorded evaluation results.

F12 — The system shall support locking a destination to a day/time and regenerating the remainder around the lock.

Verification: The lock persists after regeneration when feasible; conflicts are surfaced (proposed handling).

F13 — The system shall present the generated itinerary as both a map and a per-day timeline.

Verification: Both views represent the same scheduled stops and order.

F14 — The system shall initialize visit-duration estimates by category and refine estimates using user corrections.

Verification: A correction can be saved and influences subsequent duration estimation.

Trip preparation

F15 — The system shall generate itinerary-based tasks with due dates for activities requiring advance booking.

Verification: Relevant tasks and due dates appear in the trip task list.

F16 — The system shall estimate per-person trip costs using admission, transport, and category-average meal costs.

Verification: An estimated cost breakdown is available for each member.

F17 — The system shall record shared expenses and calculate settlement transfers intended to minimize the number of payments.

Verification: Given a sample expense ledger, balances reconcile and a settlement plan is displayed.

4. Interfaces and Data

4.1 User interface

Mobile-responsive member screens: destination discovery/preferences, pairwise comparison, group results, map, day timeline, task list, expense ledger, and packing list. Organizer-only screens: trip configuration, membership, schedule constraints, overrides and regeneration. The packing list is mentioned in the proposal interface but has no numbered functional requirement.

4.2 External interfaces

Interface	Expected use
Foursquare / OSM Overpass	Place catalog, coordinates, categories and available hours.
Wikivoyage / Wikidata	Descriptive destination information.
OSRM	Precomputed travel-time matrix from OSM road network.
OpenTripPlanner + GTFS	Optional transit routes for seed cities.
Google OR-Tools	Constraint-based itinerary optimization.
Venue websites	External booking links; no in-app checkout.

4.3 Conceptual data entities (proposed)

Entity	Core fields / relationships
User	User ID, email, authentication information.
Trip	Trip ID, organizer, city, dates, home base, daily windows.
Membership	Trip, user, role, invitation status.

Destination	Place ID, city, location, category, hours, cost, duration.
Preference	Member, destination, point allocation/rank, veto.
Comparison	Member, destination A/B, chosen destination, timestamp.
ItineraryStop	Trip, day, destination, start/end, locked flag.
Task / Expense	Trip-linked due date/status; payer, amount, beneficiaries.

5. Nonfunctional Requirements

The proposal defines architectural and usability constraints but does not specify measurable service-level targets. The following distinguish supported requirements from suggested targets.

Area	Requirement / verification	Status
Usability	Responsive UI remains usable at mobile viewport widths; check key workflows on mobile and desktop.	Source-backed
Compatibility	Supports modern JavaScript-enabled browsers.	Source-backed
Optimization	Precompute/cache travel times and enforce solver time limit; return best feasible result found.	Source-backed

Data quality	Handle incomplete or malformed opening hours defensively; curate seed-city records.	Source-backed
Security	Protect accounts and restrict trip management to authorized users; define invite-link access rules.	Proposed
Reliability	Report infeasible itineraries, missing travel times, and unavailable place data clearly.	Proposed
Performance	Specify acceptable response times, comparison latency, and optimizer time limit.	TBD
Privacy	Define data retention, account deletion, and handling of member preferences/expenses.	TBD
Maintainability	Use GitHub feature branches, reviewed PRs, and CI/CD.	Source-backed

6. Use Cases and Acceptance Tests

Use case	Primary flow / pass condition

UC-01 Create group trip	Organizer signs in, enters trip settings, and saves a trip.
UC-02 Join and vote	Member joins by link, allocates points, and records pairwise choices.
UC-03 Build group ranking	Scores are generated, weighted results shown alongside raw counts, and vetoes excluded.
UC-04 Generate itinerary	System creates a schedule within time windows and travel constraints.
UC-05 Regenerate after lock	Lock is retained and remaining stops are recalculated or a conflict is reported.
UC-06 Prepare and settle	Tasks/costs appear; expenses yield reconciled member balances and settlement suggestions.
UC-07 Compare solvers	Both scheduling methods run against identical inputs and report quality/runtime.

7. Architecture and Verification Strategy

React + Vite provides the browser interface; MapLibre GL provides maps. FastAPI exposes trip, preference, schedule, and expense operations. PostgreSQL/PostGIS stores users, destinations and trip state. NumPy/SciPy supports preference scoring; a heuristic and OR-Tools CP-SAT implement itinerary generation. Cached OSRM/OTP travel matrices are consumed during optimization. Docker Compose runs the services on one host.

Testing should include unit tests for preference scoring, point caps, hard-no filtering, expense balancing, and schedule constraints; integration tests for APIs, database, routing cache and solver; and end-to-end tests of mobile voting and itinerary viewing. For the final report, compare the heuristic and solver on identical scenarios using group

score, feasibility, fairness measure and runtime. Numerical thresholds remain to be selected by the team.

8. Open Questions / Decisions Required

- Define the fixed point budget, weighting cap, and whether ranking and points are alternatives or simultaneous inputs.
- Specify the fairness objective, score normalization, and how worst-off members are measured.
- Resolve F07 top-pick guarantee when a top pick is vetoed, closed, outside time limits, or conflicts with another mandatory stop.
- Specify who may lock itinerary items (F12 says a member; organizer controls are described separately).
- Set the minimum comparison count and behavior when a member does not finish voting.
- Define source and freshness of admission prices, meal estimates, opening hours and advance-booking due dates.
- Specify travel mode selection and handling of missing route or opening-hour data.
- Clarify packing-list functionality and whether it should receive its own requirement ID.
- Specify user authentication mechanism, invite security, access control and retention policy.
- Set concrete performance targets, solver timeout, supported browsers, and seed cities.

9. Traceability and Scope Notes

Requirements F01–F04, F06–F17 and F07a/F08a are transcribed and elaborated from Section 3.1 of the proposal. F05 is absent from the original numbering and has not been invented. The proposal also references invitations, packing lists, external venue links and member management without separate numbered requirements. Proposed clarifications in this SRS should be approved before implementation.